

```

# src/application/dto.py

from typing import Optional

from pydantic import BaseModel, Field

class UrlDTO(BaseModel):
    _id: Optional[str]
    url: str = Field(...,
pattern=r"^https:\\/\\/ (www\\.)? (youtube\\.com|youtu\\.be) \\/")

class TranscriptionDTO(UrlDTO):
    transcript: str

class SummaryQueuedDTO(UrlDTO):
    task_id: str
    task_message: str

class SummaryCompletedDTO(UrlDTO):
    summary: str
# src/application/logging_config.py

import logging

def setup_logging():
    logging.basicConfig(
        level=logging.INFO,
        format='%(asctime)s %(levelname)s %(name)s: %(message)s',
        handlers=[
            logging.StreamHandler()
        ]
    )
# # src/application/mappers.py

# from typing import Optional
# from src.domain.entities import VideoResponse, VideoURL
# from src.domain.model_exceptions import IncompleteError, UniqueIDError
# from src.application.dto import SummaryCompletedDTO, SummaryQueuedDTO

# class DTOMapper:
#     @staticmethod
#     def to_domain_url(url:str, id:Optional[str] = None)->VideoURL:
#         return VideoURL(_id=id, url=url)

#     @staticmethod
#     def to_summary_DTO(summary: VideoResponse)->SummaryCompletedDTO:
#         if summary._id is None:
#             raise UniqueIDError("Transcription is missing a primary key")
#         if summary.summaries.summary is None:
#             raise IncompleteError("Failed to generate summary")
#         return SummaryCompletedDTO(
#             id=summary._id,
#             url=summary.url,
#             summary=summary.summaries.summary
#         )

#     @staticmethod
#     def to_queued_dto(url: str, task_id: str)->SummaryQueuedDTO:
#         return SummaryQueuedDTO(
#             id=None,

```

```

#         url=url,
#         task_id=task_id,
#         task_message="Processing the video"
#     )
# src/application/task_maintenance.py

import logging
from typing import Any, AsyncIterator, Optional

from src.background.celery_app import celery_app
from src.infrastructure.mongo_service import MongoService
from src.infrastructure.redis_client import RedisClient, get_redis_client

logger = logging.getLogger(__name__)

class TaskMaintenanceService:
    def __init__(self, mongo: MongoService, redis_client:
Optional[RedisClient] = None):
        self.mongo = mongo
        self.redis = redis_client or get_redis_client()

    async def iter_queued_tasks(self) -> AsyncIterator[dict[str, Any]]:
        """Yield task documents with status QUEUED. MongoService should
expose an iterator wrapper."""
        async for doc in self.mongo.iter_tasks_by_status("QUEUED"):
            yield doc

    async def requeue_stuck_tasks(self) -> None:
        """
        Requeue stuck tasks safely:
        - use redis to avoid duplicates
        - use celery_app.AsyncResult to check broker state
        - update mongo via MongoService methods, not direct collection access
        """

        async for doc in self.iter_queued_tasks():
            try:
                if not doc or "video_url" not in doc or "task_id" not in doc:
                    logger.warning(
                        "Invalid task doc, marking INVALID: %s",
doc.get("_id")
                    )
                if id := doc.get("_id"):
                    await self.mongo.mark_task_invalid(id)
                    continue
                else:
                    logger.error(
                        "Doc missing _id, cannot mark requeue error: %s
in mark_task_invalid block",
                        doc,
                    )
                url = doc["video_url"]
                old_task_id = doc["task_id"]

                # Skip if Redis already has queued task for this url
                existing = await self.redis.get_queued_task(url)
                if existing:
                    logger.info(
                        "Redis already has queued task for %s -> %s,
skipping",
                        url,
                        existing,
                    )
            except Exception as e:
                logger.error("Error in requeue_stuck_tasks: %s", e)

```

```

        if existing != old_task_id:
            if id := doc.get("_id"):
                await self.mongo.update_task_id(id, existing)
            else:
                logger.error(
                    "Doc missing _id, cannot mark requeue
error: %s in update_task_id block",
                    doc,
                )
            continue

        # Check Celery broker state
        res = celery_app.AsyncResult(old_task_id)
        if getattr(res, "state", None) in ("PENDING", "RETRY",
"STARTED"):
            logger.info(
                "Celery indicates task %s is in %s; skipping",
                old_task_id,
                res.state,
            )
            continue

        # Send new task
        new_task = celery_app.send_task(
            "summarize_video_task",
            args=[url],
            kwargs={},
            task_id=old_task_id, # reuse same task_id to avoid
duplicates
        )
        new_task_id = getattr(new_task, "id", None) or str(new_task)

        # Try set redis queued key (set_queued_task should be atomic
NX)
        await self.redis.set_queued_task(taskID=new_task_id, url=url)

        # Update mongo task doc to reflect new task id
        if id := doc.get("_id"):
            await self.mongo.update_task_after_requeue(id,
new_task_id)
        else:
            logger.error(
                "Doc missing _id, cannot mark requeue error: %s in
update_task_after_requeue block",
                doc,
            )

            logger.info("Requeued %s as %s", url, new_task_id)

    except Exception as e:
        logger.exception("Failed to requeue doc %s: %s",
doc.get("_id"), e)
        # Optionally mark doc as needing manual attention
        try:
            if id := doc.get("_id"):
                await self.mongo.mark_task_requeue_error(id)
            else:
                logger.error(
                    "Doc missing _id, cannot mark requeue error: %s
in exception block",
                    doc,
                )
        except Exception:
            logger.exception(

```

```

        "Marking requeue error failed for %s", doc.get("_id")
    )
# src/application/use_case.py

import logging
from typing import Optional

import redis

from src.background.celery_app import celery_app
from src.domain.entities import TaskStatusResponse, VideoResponse, VideoURL
from src.domain.model_exceptions import InsufficientData, TaskIDError
from src.infrastructure.mongo_service import MongoService
from src.infrastructure.redis_client import RedisClient
from src.repo.video_repository import VideoRepository

logger = logging.getLogger(__name__)

class UseCase:
    _video_rep: VideoRepository
    _redis_client: RedisClient
    _mongo_client: MongoService

    def __init__(
        self, vid_repo: VideoRepository, r_client: RedisClient, m_client:
MongoService
    ) -> None:
        self._video_rep = vid_repo
        self._redis_client = r_client
        self._mongo_client = m_client

    async def _get_from_redis(self, video: VideoURL) ->
Optional[VideoResponse]:
        """check if the video is already cached in redis"""
        logger.warning("fetching from redis")
        response = await self._redis_client.get_cached_summary(url=video.url)
        if response is not None:
            logger.info("Cache hit")
        else:
            logger.info("Cache miss")
        return response

    async def _get_from_db(self, video: VideoURL) -> Optional[VideoResponse]:
        """check if the video is already saved in db"""
        logger.info("fetching from database")
        response = await self._video_rep.get(video_url=video, _id=None)
        if response:
            logger.info("Database hit")
            logger.info(f"caching the summary of: {response.url}")
            await self._redis_client.set_cache_summary(response)
        else:
            logger.info("Database miss")
        return response

    async def _check_if_queued(
        self, task_id: Optional[str], video: Optional[VideoURL]
    ) -> Optional[dict]:
        """check if the video is already queued for Celery"""
        if task_id:
            return await self._mongo_client.get_status(task_id=task_id,
video_url=None)
        elif video:
            return await self._mongo_client.get_status(

```

```

        task_id=task_id, video_url=video.url
    )
    else:
        raise InsufficientData("The request is missing the required
informations")

    async def _combine_redis_db(self, video: VideoURL) ->
Optional[VideoResponse]:
        vid_response: Optional[VideoResponse] = None
        vid_response = await self._get_from_redis(video)
        if vid_response is None:
            vid_response = await self._get_from_db(video)
        return vid_response

    async def send(
        self, video: Optional[VideoURL] = None, task_id: Optional[str] = None
    ) -> VideoResponse | TaskStatusResponse:
        try:
            is_queued = await self._check_if_queued(task_id=task_id,
video=video)
            if is_queued:
                queued_status = is_queued.get("status", None)
                queued_task_id = is_queued.get("task_id", None)
                if not queued_status or not queued_task_id:
                    logger.warning(
                        f"Task status record for task_id {task_id} is missing
'status' or 'task_id' fields"
                    )
                    raise RuntimeError(
                        f"Invalid task status record for task_id {task_id}"
                    )
                if queued_status in ("QUEUED", "STARTED"):
                    return TaskStatusResponse(
                        task_id=queued_task_id,
                        status=queued_status,
                        message="Video already queued for processing",
                    )
                elif queued_status in ["TIMEOUT", "FAILED"]:
                    raise RuntimeError(
                        f"Previous task {queued_task_id} failed or timed out:
{queued_status}"
                    )
                elif queued_status == "SUCCESS":
                    video = VideoURL(_id=None, url=is_queued["video_url"])
            if video:
                redis_task = await
self._redis_client.get_queued_task(url=video.url)
                if redis_task:
                    return TaskStatusResponse(
                        task_id=redis_task,
                        message="Video already queued for processing",
                    )
                response = await self._combine_redis_db(video)
                if response is None:
                    from uuid import uuid4

                    new_task_id = str(uuid4())

                    claimed_task = await self._redis_client.set_queued_task(
                        taskID=new_task_id, url=video.url
                    )
                    if not claimed_task:
                        old_task_id = await
self._redis_client.get_queued_task(

```

```

        url=video.url
    )
    if old_task_id is None:
        logger.warning(
            f"Failed to claim queue for {video.url} and
no existing task found in Redis"
        )
        raise RuntimeError(
            f"Failed to claim queue for {video.url} and
no existing task found in Redis"
        )
        logger.info(
            f"Another task already queued for {video.url}
with taskID {claimed_task}"
        )
        return TaskStatusResponse(
            task_id=old_task_id,
            message="Video already queued for processing",
        )

    await self._mongo_client.insert_task_status(new_task_id,
video.url)

    try:
        task = celery_app.send_task(
            "summarize_video_task",
            args=[video.url],
            task_id=new_task_id,
        )
    except Exception as e:
        logger.exception(f"Failed to send Celery task: {e}")
        await
self._redis_client.delete_queued_task(video.url)
        raise Exception("Failed to start the distributed
task") from e

    return TaskStatusResponse(
        task_id=task.id,
        message="Video is being processed",
    )
    return response

    raise TaskIDError("Incorrect data, please provide a valid ID")
except redis.RedisError as re:
    logger.error(f"Redis error: {re}")
    raise Exception("Cache error")
except Exception as e:
    logger.error(f"UseCase send failed: {e}")
    raise
# src/application/video_pipeline_service.py

from typing import Optional
import logging
from src.domain.entities import SummaryResponse, VideoResponse
from src.domain.model_exceptions import SummaryFailException,
VideoNotAvailableError
from src.infrastructure.correction_service import Correction_Service
from src.infrastructure.stt_service import STTService
from src.infrastructure.summarizer_service import SummarizerService
from src.infrastructure.yt_service import YoutubeService

logger = logging.getLogger(__name__)

class VideoPipelineService:
    def __init__(self,

```

```

        yt_service: YoutubeService,
        stt_service: STTService,
        c_service: Correction_Service,
        s_service: SummarizerService
    ) -> None:
        self._yt_service = yt_service
        self._stt_service = stt_service
        self._c_service = c_service
        self._s_service = s_service

    def _get_captions(self, url: str) -> Optional[str]:
        logger.debug("Fetching Captions")
        captions = self._yt_service.download_captions(url=url)
        return captions

    async def _get_transcription(self, url: str) -> str:
        logger.debug(f"Downloading audio: {url}")
        chunks = await self._yt_service.download(url)
        logger.debug(f"Transcribing: {url}")
        transcription = await self._stt_service.transcribe_audio(chunks)
        return transcription

    def _correct_grammar(self, transcription: str) -> str:
        logger.debug("\n--- Corrected Transcription ---\n")
        corrected_transcription = self._c_service.clean(transcription)
        return corrected_transcription

    def _generate_summary(self, c_transcription: str) -> str:
        logger.debug("\n--- Generate Summary ---\n")
        prompt = f"""
            Summarize the following transcription exactly as it is.

            Rules:
            1. Only use sentences or phrases that appear in the
transcription.
            2. Do NOT include any social media mentions, emails, calls-to-
action, or invented content.
            3. Do NOT mention YouTube, Twitter, Facebook, blogs, or any
external platform.
            4. Keep the summary factual, concise, and limited to the content.
            5. Ignore any phrases like 'hit subscribe', 'share this video',
or 'contact me'.

            Output only the summary. No extra commentary.

            Transcription:
            {c_transcription}
        """
        summary = self._s_service.summarize(prompt)
        return summary

    async def transform_the_video(self, url: str) -> VideoResponse:
        captions = self._get_captions(url)
        if captions is None or captions == "":
            transcription = await self._get_transcription(url)
            if not transcription:
                raise VideoNotAvailableError(f"Cannot get captions or
transcription for {url}")
            captions = self._correct_grammar(transcription)
            if not captions:
                raise ValueError(f"Could not get captions or transcription for
{url}")

            summary = self._generate_summary(captions)

```

```

        if not summary:
            raise SummaryFailException(f"Failed to generate summary for
{url}")

        video_response = VideoResponse(
            _id=None,
            url=url,
            transcription=captions,
            summaries=SummaryResponse(
                summary=summary,
                model_name="google/flan-t5-large",
                latest=True,
                created_at=None
            ),
            created_at=None # DB will populate this
        )
        return video_response
# src/auth/base.py

from typing import Optional

from authlib.integrations.starlette_client import OAuth
from fastapi import Request
from fastapi.security import OAuth2PasswordBearer
from pydantic import BaseModel, model_validator
from src.repo.user_repo import get_user_repo

from src.auth.constants import TokenData
from src.auth.tokens import get_token_service

oauth = OAuth()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/token")

class OAuthProvider(BaseModel):
    name: str
    client_id: str
    client_secret: str
    server_metadata_url: Optional[str] = None
    client_kwargs: Optional[dict] = None

    @model_validator(mode="before")
    @classmethod
    def validate_content(cls, values):
        field_names = ["name", "client_id", "client_secret"]

        def check_empty_field(field_name: str) -> None:
            value = values.get(field_name, "")
            if value.strip() == "":
                raise ValueError(f"{field_name.replace('_', ' ').title()} is
required")

        for field_name in field_names:
            check_empty_field(field_name)
        return values

def register_oauth(providers: list[OAuthProvider]) -> None:
    """To be called one time in app lifespan to register OAuth providers."""
    for provider in providers:
        oauth.register(
            name=provider.name,
            client_id=provider.client_id,
            client_secret=provider.client_secret,

```

```

        server_metadata_url=provider.server_metadata_url,
        client_kwargs=provider.client_kwargs,
    )

class AuthBase:
    def __init__(self, secret_key: str):
        self.secret_key = secret_key
        self.token_service = get_token_service(secret_key)
        self.user_repo = get_user_repo()

class OAuthBase(AuthBase):
    def __init__(self, secret_key: str):
        super().__init__(secret_key)

    async def initiate_oauth(self, request: Request, redirect_url: str) ->
None: ...
    async def handle_callback(self, request: Request) -> TokenData: ...
# src/auth/constants.py

from typing import Literal, Optional

from fastapi import HTTPException, status
from pydantic import BaseModel

ACCESS_TOKEN_EXP = 15 * 60 # 15 minutes
REFRESH_TOKEN_EXP = 7 * 24 * 60 * 60 # 7 days
TOKEN_TYPE = Literal["access", "refresh"]

class TokenData(BaseModel):
    access_token: str
    refresh_token: Optional[str] = None
    token_type: str = "bearer"

class CredentialsException(HTTPException):
    def __init__(self, detail: str = "Could not validate credentials"):
        super().__init__(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail=detail,
            headers={"WWW-Authenticate": "Bearer"},
        )
# src/auth/exceptions.py

from typing import Optional

class AuthException(Exception):
    """Base class for authentication-related exceptions."""

    pass

class FailedToFetchUserInfoException(AuthException):
    """Raised when fetching user info from the provider fails."""

    def __init__(self, provider: str, original_exception: Optional[Exception]
= None):
        super().__init__(
            f"Failed to fetch user info from {provider}"
            + (f": {str(original_exception)}" if original_exception else "")

```

```

    )

class MissingProviderSubException(AuthException):
    """Raised when the 'sub' field is missing in the provider's user info."""

    def __init__(self, provider: str):
        super().__init__(f"Missing 'sub' field in {provider} user info.")

class UserNotFoundException(AuthException):
    """Raised when a user is not found in the database."""

    def __init__(self, user_id: str):
        super().__init__(f"User with ID {user_id} not found.")

class InvalidCredentialsException(AuthException):
    """Raised when provided credentials are invalid."""

    def __init__(self):
        super().__init__("Invalid username or password.")

class TokenExpiredException(AuthException):
    """Raised when a token has expired."""

    def __init__(self):
        super().__init__("Token has expired.")

class TokenInvalidException(AuthException):
    """Raised when a token is invalid."""

    def __init__(self):
        super().__init__("Token is invalid.")

class OAuthException(AuthException):
    """Raised when an OAuth-related error occurs."""

    def __init__(self, provider: str, message: str):
        super().__init__(f"OAuth error with {provider}: {message}")
# src/auth/providers.py

import logging
from functools import lru_cache

from fastapi import Request

from src.auth.base import OAuthBase, oauth
from src.auth.constants import TokenData
from src.auth.exceptions import (
    FailedToFetchUserInfoException,
    MissingProviderSubException,
    OAuthException,
)
from src.models.user import ProviderInfo, UserDB

logger = logging.getLogger(__name__)

class GoogleAuth(OAuthBase):

```

```

def __init__(
    self,
    secret_key: str,
    google_client_id: str,
    google_client_secret: str,
):
    super().__init__(secret_key)
    self.g_c_i = google_client_id
    self.g_c_s = google_client_secret

    async def initiate_oauth(
        self,
        request: Request,
        redirect_url: str,
    ) -> None:
        return await oauth.google.authorize_redirect(request, redirect_url)

    async def handle_callback(self, request: Request) -> TokenData:
        try:
            token = await oauth.google.authorize_access_token(request)
            user_info = token.get("userinfo", {})
            if not user_info:
                raise FailedToFetchUserInfoException("Google")
            provider_user_id = user_info.get("sub", None)
            if not provider_user_id:
                raise MissingProviderSubException("Google")
            user_email = user_info.get("email", None)
            if not user_email:
                raise FailedToFetchUserInfoException("Google")
            user_obj = UserDB(
                provider=ProviderInfo(
                    name="google",
                    user_id=provider_user_id,
                ),
                email=user_email,
                username=user_email.split("@")[0], # use name before @
            )
            default_username = user_obj.username
            user = await self.user_repo.find_or_create_user(user_obj)
            self.tokens = self.token_service.login(str(user.id))
            return self.tokens
        except (FailedToFetchUserInfoException, MissingProviderSubException):
            raise
        except Exception as e:
            logger.error(f"Error handling Google OAuth callback: {str(e)}")
            raise OAuthException(
                "Google", f"Google OAuth callback handling failed: {str(e)}"
            )

    @lru_cache()
    def get_google_auth(
        self,
        secret_key: str,
        google_client_id: str,
        google_client_secret: str,
    ) -> GoogleAuth:
        return GoogleAuth(secret_key, google_client_id, google_client_secret)
    # src/auth/tokens.py

from datetime import datetime, timedelta, timezone
from functools import lru_cache
from uuid import uuid4

import jwt

```

```

from fastapi import HTTPException
from src.auth.constants import (
    ACCESS_TOKEN_EXP,
    REFRESH_TOKEN_EXP,
    TOKEN_TYPE,
    CredentialsException,
    TokenData,
)

class TokenService:
    """
    Service for generating and verifying JWT tokens.
    This class provides methods to create access and refresh tokens,
    as well as to verify and refresh tokens.
    """

    def __init__(self, secret_key: str, algorithm: str = "HS256"):
        self._secret_key = secret_key
        self._algorithm = algorithm

    def _generate_token(self, user_id: str, exp: int, token_type: TOKEN_TYPE)
-> str:
        now = datetime.now(timezone.utc)
        end_time = now + timedelta(seconds=exp)
        payload = {
            "jti": str(uuid4()),
            "sub": user_id,
            "token_type": token_type,
            "iat": int(now.timestamp()),
            "exp": int(end_time.timestamp()),
        }
        try:
            encoded_token = jwt.encode(
                payload, self._secret_key, algorithm=self._algorithm
            )
        except Exception as e:
            raise RuntimeError(f"Token generation failed: {str(e)}")
        return encoded_token

    def verify_token(self, token: str, expected_type: TOKEN_TYPE = "access")
-> dict:
        try:
            payload = jwt.decode(token, self._secret_key,
algorithms=[self._algorithm])
            user_id: str = payload.get("sub", "")
            if user_id == "":
                raise CredentialsException
            if payload.get("token_type") != expected_type:
                raise CredentialsException("Invalid token type")
        except jwt.ExpiredSignatureError:
            raise CredentialsException
        except jwt.InvalidTokenError:
            raise CredentialsException
        return payload

    def _create_access_token(self, user_id: str) -> str:
        return self._generate_token(user_id, exp=ACCESS_TOKEN_EXP,
token_type="access")

    def _create_refresh_token(self, user_id: str) -> str:
        return self._generate_token(
            user_id, exp=REFRESH_TOKEN_EXP, token_type="refresh"
        )

```

```

def new_access_token_from_refresh_token(self, token: str) -> TokenData:
    try:
        payload = self.verify_token(token, expected_type="refresh")
        user_id: str = payload.get("sub", "")
        if user_id == "":
            raise CredentialsException("Invalid token")
        return self.login(user_id)
    except HTTPException:
        raise
    except Exception as e:
        raise CredentialsException(f"Token refresh failed: {str(e)}")

def login(self, user_id: str) -> TokenData:
    access_token = self._create_access_token(user_id)
    refresh_token = self._create_refresh_token(user_id)
    return TokenData(access_token=access_token,
refresh_token=refresh_token)

@lru_cache()
def get_token_service(secret_key: str) -> TokenService:
    return TokenService(secret_key)
# src/background/celery_app.py

from celery import Celery
from src.infrastructure.system_config import config

celery_app = Celery(
    "yt-service",
    broker=config.CELERY_BROKER_URL,
    backend=config.CELERY_BACKEND_URL,
    include=["src.background.celery_task"]
)

celery_app.conf.update(
    task_acks_late=True,
    task_reject_on_worker_lost=True,
    task_serializer='json',
    accept_content=['json'],
    result_serializer='json',
    timezone='UTC',
    enable_utc=True,
)

from src.background import celery_task
# src/background/celery_task.py

import asyncio
import logging

from billiard.exceptions import SoftTimeLimitExceeded

from src.application.video_pipeline_service import VideoPipelineService
from src.background.celery_app import celery_app
from src.domain.model_exceptions import SoftTimeLimitExceededError
from src.infrastructure.correction_service import get_correction_service
from src.infrastructure.mongo_service import MongoService
from src.infrastructure.redis_client import get_redis_client
from src.infrastructure.stt_service import STTService
from src.infrastructure.summarizer_service import SummarizerService
from src.infrastructure.system_config import config
from src.infrastructure.video_repository_imp import VideoRepositoryImp
from src.infrastructure.yt_service import YoutubeService

```

```

logger = logging.getLogger(__name__)

@celery_app.task(
    bind=True,
    name="summarize_video_task",
    max_retries=3,
    default_retry_delay=60,
    ignore_result=False,
)
def queue_yt_video(self, url: str):
    """
    Celery task to summarize a YouTube video.
    This function runs synchronously but executes async code inside an event
    loop.
    """

    logger.info("Entered the Celery queue")
    redis_client = get_redis_client(config.REDIS_URL)

    async def run_pipeline():
        logger.info("Running the asynchronous run_pipeline method")
        # Initialize services
        yt_service = YoutubeService()
        stt_service = STTService()
        correction_service = get_correction_service()
        summarizer_service = SummarizerService()
        mongo = MongoService(uri=config.DATABASE_URL)
        await mongo.run_init()

        video_repo = VideoRepositoryImp(mongo_service=mongo)

        await mongo.update_status(self.request.id, "STARTED")
        try:
            # the actual video processing pipeline
            vid_pipeline = VideoPipelineService(
                yt_service,
                stt_service,
                correction_service,
                summarizer_service,
            )

            video_response = await vid_pipeline.transform_the_video(url=url)

            saved_response = await video_repo.save(video_response)
            await redis_client.set_cache_summary(saved_response)
            await mongo.update_status(self.request.id, "SUCCESS")

            return self.request.id

        except SoftTimeLimitExceeded:
            logger.exception(f"Task {self.request.id} timed out for URL {url}")
            await mongo.update_status(self.request.id, "TIMEOUT")
            raise SoftTimeLimitExceededError(
                f"Task {self.request.id} timed out for URL {url}"
            )
        except Exception as e:
            logger.exception(
                f"Task {self.request.id} failed out for URL {url} for: {e}"
            )
            await mongo.update_status(self.request.id, "FAILED")
            raise

```

```

        finally:
            await redis_client.delete_queued_task(url)

    try:
        return asyncio.run(run_pipeline())
    except Exception as e:
        logger.error(f"Celery task {self.request.id} failed: {e}")
        raise Exception("failed to start the distributed task")
# src/db/base.py

import logging
from typing import Optional, ParamSpec, TypeVar

from motor.motor_asyncio import AsyncIOMotorCollection

from src.db.helper import IndexConfig
from src.infrastructure.mongo_service import handle_mongo_exception

logger = logging.getLogger(__name__)

P = ParamSpec("P")
R = TypeVar("R")

class BaseRepo:
    def __init__(self, collection: AsyncIOMotorCollection):
        self._collection = collection

    @handle_mongo_exception
    async def ensure_indexes(
        self, field_indices: Optional[list[IndexConfig]] = None
    ) -> None:
        if not field_indices:
            return # No indexes to create

        for config in field_indices:
            index_spec, options = config.to_index_spec()
            await self._collection.create_index(
                index_spec,
                **options,
            )
            logger.info(f"Index '{options.get('name')}' created successfully.")
# src/db/helper.py

from dataclasses import dataclass
from typing import NotRequired, TypedDict, Union

from pymongo import ASCENDING

class IndexOptions(TypedDict):
    unique: NotRequired[bool]
    sparse: NotRequired[bool]
    name: NotRequired[str]

@dataclass
class IndexConfig:
    key: str
    order: Union[int, str] = ASCENDING
    unique: bool = False
    sparse: bool = False
    index_name: str = ""

```

```

    def to_index_spec(
        self,
    ) -> tuple[list[tuple[str, Union[int, str]]], IndexOptions]:
        """Converts the IndexConfig to a format suitable for MongoDB index
        creation."""
        order_str = str(self.order).replace(" ", "_")
        name = (
            self.index_name
            or f"{self.key}_{order_str}_{'unique' if self.unique else 'idx'}"
        )

        return [(self.key, self.order)], {"unique": self.unique, "name":
name}
# src/db/mongo_manager.py

import asyncio
import logging
from typing import Annotated, Optional

from motor.motor_asyncio import AsyncIOMotorClient, AsyncIOMotorDatabase
from pydantic import Field
from pymongo.errors import (
    PyMongoError,
)

from src.domain.model_exceptions import DatabaseUnavailableError

logger = logging.getLogger(__name__)

class MongoManager:
    """
    Manages MongoDB connection and provides utility methods for database
    operations.
    """

    def __init__(self, uri: str, db_name: str) -> None:
        self._uri = uri
        self._db_name = db_name
        self._client: Optional[AsyncIOMotorClient] = None
        self._db_internal: Optional[AsyncIOMotorDatabase] = None
        self._startup_lock = (
            asyncio.Lock()
        ) # lock to ensure connect is only called once
        self._collection = None # collection for user location data

    async def _connect_internal(self) -> None:
        """Internal method to establish MongoDB connection. Should be called
        once at startup."""
        async with self._startup_lock:
            if self._client:
                return # already connected
            try:
                self._client = AsyncIOMotorClient(
                    host=self._uri,
                    serverSelectionTimeoutMS=5000,
                    connectTimeoutMS=5000,
                    socketTimeoutMS=5000,
                )
                self._db_internal = self._client[self._db_name]
                await self.health_check()
                return # connection successful
            except Exception as e:

```

```

        logger.warning(f"Connection failed: {e}")
        self._client = None
        self._db_internal = None
        raise DatabaseUnavailableError(
            "An unexpected error occurred while connecting to the
database."
        ) from e

    async def connect(self) -> None:
        """Establishes the MongoDB connection."""
        await self._connect_internal()

    async def close(self) -> None:
        """Closes the MongoDB connection."""
        if self._client:
            self._client.close()
            self._client = None
            self._db_internal = None

    async def health_check(self) -> bool:
        """Performs a health check by pinging the MongoDB server."""
        if self._client is None:
            return False
        try:
            await self._client.admin.command("ping")
            return True
        except PyMongoError as e:
            logger.error(f"MongoDB health check failed: {e}")
            raise DatabaseUnavailableError() from e

    @property
    def _db(self) -> AsyncIOMotorDatabase:
        """Returns the database instance, ensuring connection is
established."""
        if self._db_internal is None:
            raise RuntimeError("Database connection not established")
        return self._db_internal

    def get_collection(self, collection_name: str):
        """Returns a collection from the database."""
        return self._db[collection_name]

MONGO_MANAGER: Annotated[
    Optional[MongoManager], Field(description="Singleton instance of
MongoManager")
] = None

def create_mongo_manager(*, uri: str, db_name: str) -> MongoManager:
    """Returns a singleton instance of MongoManager."""
    global MONGO_MANAGER
    if MONGO_MANAGER is None:
        MONGO_MANAGER = MongoManager(uri, db_name)
    return MONGO_MANAGER

def get_mongo_manager() -> MongoManager:
    """Returns the singleton instance of MongoManager, ensuring it is
initialized."""
    if MONGO_MANAGER is None:
        raise RuntimeError(
            "MongoManager has not been initialized. Call create_mongo_manager
first."

```

```

    )
    return MONGO_MANAGER
# src/domain/entities.py

from datetime import datetime
from typing import Optional

from pydantic import BaseModel, Field

class VideoURL(BaseModel):
    _id: Optional[str]
    url: str = Field(...,
pattern=r"^https://(www\.)?(youtube\.com|youtu\.be)/")

class SummaryResponse(BaseModel):
    summary: str
    model_name: str
    latest: bool
    created_at: Optional[datetime]

class VideoResponse(VideoURL):
    transcription: str
    summaries: SummaryResponse
    created_at: Optional[datetime]

class TaskStatusResponse(BaseModel):
    task_id: str = Field(..., description="Unique identifier for the task")
    status: Optional[str] = Field(
        default=None,
        description="Status of the task. Can be 'queued', 'processing',
'completed', 'failed', or None if not found.",
    )
    message: Optional[str] = Field(
        default=None,
        description="Additional information about the task status, such as
error messages if the task failed.",
    )
# src/domain/model_exceptions.py

class DatabaseUnavailableError(Exception):
    """Raised when the database is unavailable or connection fails."""

    def __init__(
        self,
        message: str = "Database is currently unavailable. Please try again
later.",
    ):
        self.message = message
        super().__init__(self.message)

class UniqueIDError(Exception):
    """Error in the primary key"""

    pass

class TaskIDError(Exception):
    """Provided incorrect task ID"""

```

```

pass

class IncompleteError(Exception):
    """If the summary was not generated"""

    pass

class VideoNotAvailableError(Exception):
    """If the URL or video does not exists"""

    pass

class InsufficientData(Exception):
    """Insufficient data is given"""

    pass

class SummaryFailException(Exception):
    """Summary generation for the video fails"""

    pass

class FailedToFetch(Exception):
    """Failed to fetch the data in db or cache"""

    pass

class FailedToSave(Exception):
    """Failed to save the data in db or cache"""

    pass

class SoftTimeLimitExceededError(Exception):
    """Soft Time Limit Exceeded"""

    pass

class RecordNotFoundError(Exception):
    """Record not found in the database"""

    def __init__(self, message: str = "Record not found in the database"):
        self.message = message
        super().__init__(self.message)
# src/infrastructure/correction_service.py

import logging
import os
from functools import lru_cache
from pathlib import Path

import language_tool_python as ltp

logger = logging.getLogger(__name__)

ignore_list = [

```

```

        "Java",
        "Python",
        "TensorFlow",
        "Hadoop",
        "Spring",
        "Django",
        "JVM",
        "NASA",
    ]
    replace_dict = {
        "Jawa": "Java",
        "Jaw": "Java",
        "Possum": "Rossum", # Guido van Rossum
        "Vista": "Guido", # if Whisper mangles his name
    }

class Correction_Service:
    def __init__(
        self,
        lang: str = "en-US",
        ignore_list: list[str] = ignore_list,
        replace_dict: dict[str, str] = replace_dict,
    ) -> None:
        """
        Initialize the correction tool.
        lang: language code (default is English, US).
        """
        logger.info("Initializing Correction_Service with LanguageTool")
        jar_dir = os.getenv("LTP_JAR_DIR_PATH")
        logger.info(f"Using LanguageTool directory: {jar_dir}")

        # Explicitly set the LanguageTool home directory
        if jar_dir:
            p = Path(jar_dir)
            if p.exists():
                logger.info(f"Using LanguageTool directory: {jar_dir}")
                os.environ["LTP_HOME"] = jar_dir
            else:
                logger.warning(
                    f"LTP_JAR_DIR_PATH set but does not exist: {jar_dir}."
                )
                os.environ.pop("LTP_HOME", None)
        else:
            # Let language_tool_python use its normal cache behavior
            os.environ.pop("LTP_HOME", None)
        try:
            self.tool = ltp.LanguageTool(lang)
            logger.info("Correction_Service initialized successfully")
        except Exception as e:
            logger.error(f"Failed to initialize LanguageTool: {str(e)}")
            raise RuntimeError(f"LanguageTool initialization failed: {str(e)}")

        self.ignore_list = ignore_list or []
        self.replace_dict = replace_dict or {}

    def clean(self, text: str) -> str:
        """
        Correct spelling and grammar in a given text.
        """
        # Step 1: Apply custom replacements before correction
        for wrong, right in self.replace_dict.items():

```

```

        text = text.replace(wrong, right)

    # To Find all issues in the text
    matches = self.tool.check(text)

    # Step 3: Filter out issues that touch ignore words
    filtered_matches = []
    for match in matches:
        if any(word in match.context for word in self.ignore_list):
            continue
        filtered_matches.append(match)

    # Correct the errors
    corrected = ltp.utils.correct(text, filtered_matches)

    return corrected

@lru_cache(maxsize=1)
def get_correction_service() -> Correction_Service:
    """
    Get a cached instance of the Correction_Service.
    """
    return Correction_Service()
# src/infrastructure/mongo_service.py

import logging
from datetime import datetime, timezone
from functools import wraps
from typing import Any, AsyncIterator, Optional, Union

from bson import ObjectId
from motor.motor_asyncio import AsyncIOMotorClient
from pymongo.errors import DuplicateKeyError, PyMongoError, WriteError

from src.domain.model_exceptions import InsufficientData

logger = logging.getLogger(__name__)

def handle_mongo_exception(func):
    @wraps(func)
    async def wrapper(*args, **kwargs):
        try:
            return await func(*args, **kwargs)
        except DuplicateKeyError as e:
            logger.error(f"Duplicate key error in {func.__name__}: {e}")
            return None
        except WriteError as e:
            logger.error(f"Write error in {func.__name__}: {e}")
            return None
        except PyMongoError as e:
            logger.error(f"MongoDB error in {func.__name__}: {e}")
            return None

    return wrapper

class MongoService:
    def __init__(self, uri: str, db_name: str = "yt_summarizer") -> None:
        """
        Connect to MongoDB using Motor.
        uri: MongoDB connection string (local or Atlas cloud).
        db_name: database name (default = yt_summarizer).

```

```

        """
        self._client = AsyncIOMotorClient(uri)
        self._db = self._client[db_name]
        self._collection = self._db["videos"]
        self._task = self._db["tasks"]

    async def run_init(self):
        """Initialize indexes"""
        await self.connect()
        await self._collection.create_index("url", unique=True)
        await self._task.create_index("video_url", unique=True)
        logger.info("Mongo indexes ensured")

    async def connect(self):
        """Test the connection (ping)."""
        try:
            await self._client.admin.command("ping")
            logger.info(
                "Pinged your deployment. You successfully connected to
MongoDB!"
            )
        except Exception as e:
            logger.exception(f"MongoDB connection error: {e}")

    def disconnect(self):
        try:
            self._client.close()
        except Exception as e:
            logger.exception(f"MongoDB disconnect error: {e}")

    @handle_mongo_exception
    async def insert_task_status(self, task_id: str, video_url: str):
        if not task_id or not video_url:
            logger.error("Cannot insert task status: missing task_id or
video_url")
            raise InsufficientData("Missing the required task_id or
video_url")
        await self._task.insert_one(
            {
                "task_id": task_id,
                "video_url": video_url,
                "status": "QUEUED",
                "created_at": datetime.now(timezone.utc),
                "updated_at": datetime.now(timezone.utc),
            }
        )

    @handle_mongo_exception
    async def update_status(self, task_id: str, status: str):
        await self._task.update_one(
            {"task_id": task_id},
            {
                "$set": {
                    "status": status,
                    "updated_at": datetime.now(timezone.utc),
                }
            },
            upsert=True,
        )

    async def get_status(
        self, task_id: Optional[str], video_url: Optional[str]
    ) -> Optional[dict]:
        if not task_id and not video_url:

```

```

        logger.error("Get Status ERROR: missing both task_id and
video_url")
        raise InsufficientData("Insufficient Data provided")
    query = {}
    if task_id:
        query["task_id"] = task_id
    if video_url:
        query["video_url"] = video_url
    return await self._task.find_one(query)

async def get_video(
    self, url: Optional[str] = None, _id: Optional[str] = None
) -> Optional[dict]:
    """
    Retrieve a video document by URL, returning only the latest summary.
    """
    if url is None and _id is None:
        raise InsufficientData("Either url or _id must be provided")
    condition = {}
    if url:
        condition = {"$match": {"url": url}}
    elif _id:
        condition = {"$match": {"_id": ObjectId(_id)}}

    pipeline = [
        condition,
        {
            "$project": {
                "_id": 1,
                "url": 1,
                "transcription": 1,
                "created_at": 1,
                "summaries": {
                    "$filter": {
                        "input": "$summaries",
                        "as": "summary", #

```